

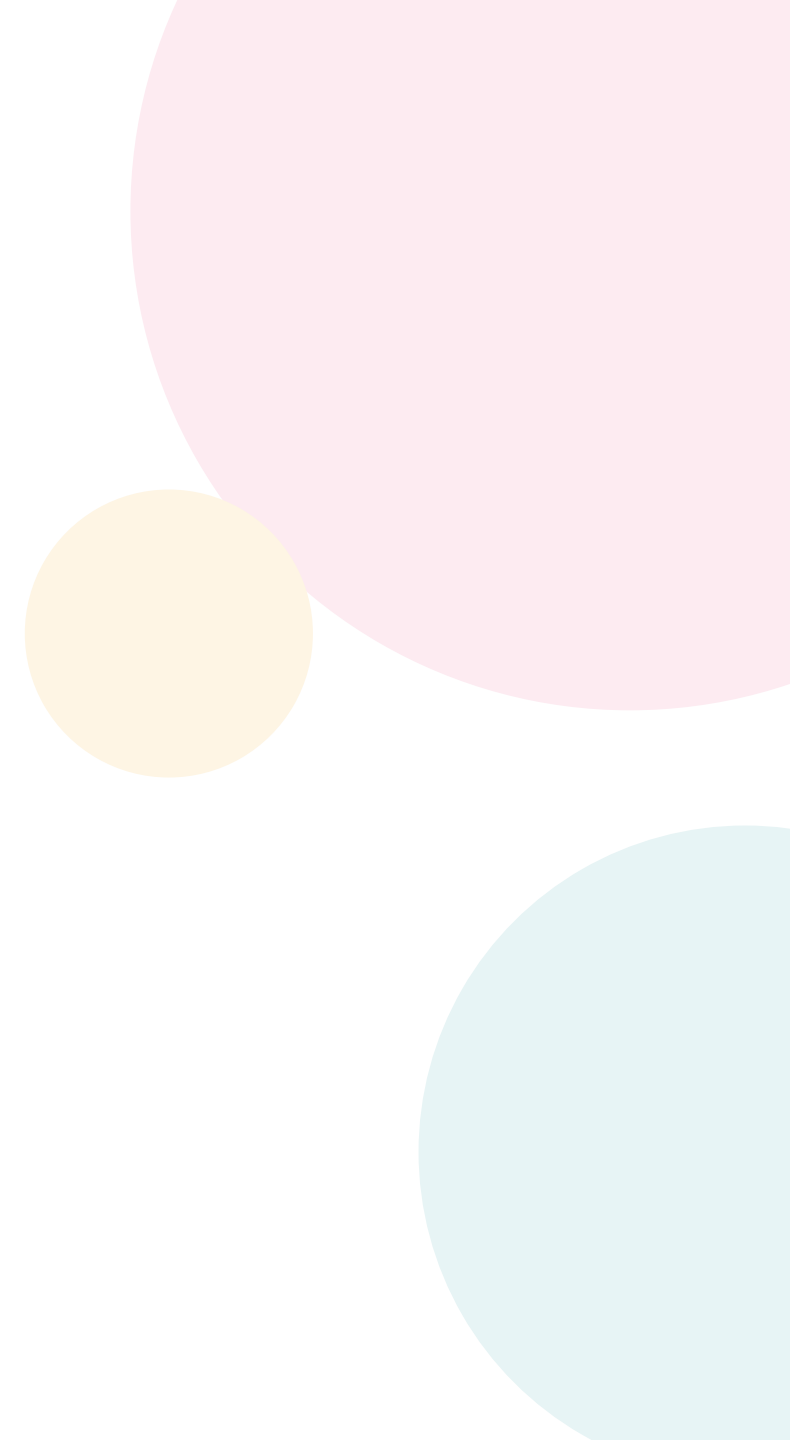
PROGRAMAÇÃO ORIENTADA A OBJETOS COM JAVA E WEB

Programação Funcional

Lambdas, interfaces funcionais e a API de Streams

Prof. Dr. Antonio Marcos Selmini

selmini@fiap.com.br · Sistemas de Informação



AGENDA

Roteiro da aula

01

O paradigma funcional em Java

02

Expressões lambda e boilerplate

03

Interfaces funcionais e method reference

04

Streams: pipeline e avaliação preguiçosa

05

IntStream e operações numéricas

06

Streams de objetos, agrupamento e exercícios

O que é programação funcional?

Paradigma que trata funções como cidadãos de primeira classe: elas podem ser passadas como argumento, retornadas por outras funções e armazenadas em variáveis.

Declarativo

Expressa o que deve ser feito, em vez de detalhar passo a passo como fazer.

Imutabilidade

Os dados originais não são alterados: cada etapa produz um novo resultado.

Sem efeitos colaterais

A função depende só das entradas e não altera o estado externo ao seu escopo.

- Em Java: lambdas, interfaces funcionais e Streams. O paradigma funcional convive com o orientado a objetos — não substitui.

PARADIGMA

Imperativo × declarativo

O mesmo problema: somar o dobro dos números pares de uma lista.

Como fazer — imperativo

```
int soma = 0;
for (int v : lista) {
    if (v % 2 == 0) {
        soma = soma + v * 2;
    }
}
```

O que fazer — declarativo

```
int soma = lista.stream()
    .filter(v -> v % 2 == 0)
    .mapToInt(v -> v * 2)
    .sum();
```

- Nos dois casos o resultado é o mesmo. No estilo declarativo você descreve a intenção — filtrar, transformar, somar — e a API cuida do laço, do índice e da variável acumuladora.

Expressão lambda

Uma **lambda expression** é uma forma concisa de representar uma **função (método) anônima** — uma função sem nome, que pode ser passada como argumento, armazenada em variáveis ou retornada por métodos.

■ sintaxe

```
(parametros) -> expressao  
(parametros) -> { bloco de codigo }
```

(parametros)

a lista de entrada, entre parênteses

->

o operador seta, que separa entrada de corpo

corpo

uma expressão simples ou um bloco entre chaves

LAMBDA

As formas de escrever uma lambda

1 Sem parâmetro

```
() -> System.out.println("Oi")
```

2 Um parâmetro

```
v -> v * 2
```

3 Vários parâmetros

```
(a, b) -> a + b
```

4 Bloco com return

```
(a, b) -> {  
    return a + b;  
}
```

- Com um único parâmetro os parênteses são opcionais. Com corpo de uma expressão, o return é implícito.

Vantagens das lambdas

1

Menos boilerplate

Dispensa classes anônimas longas para tarefas simples.

2

Streams API

É a porta de entrada para filter, map, sorted e collect.

3

Estilo declarativo

Aproxima o Java de linguagens como Haskell e Scala.

4

Mais legível

O código descreve a intenção, não a mecânica do laço.

- A lambda não cria uma linguagem nova: ela é a implementação, escrita em uma linha, do único método de uma interface funcional.

O que é boilerplate?

Chamamos de boilerplate o código repetitivo e padronizado que precisamos escrever sempre, mas que não acrescenta lógica nova ao programa — ele serve apenas para cumprir regras da linguagem ou da API.

- Boilerplate é código obrigatório, repetitivo e burocrático, que não carrega lógica essencial do programa.

Por que reduzir boilerplate é importante?

- ✓ Torna o código mais legível e direto
- ✓ Evita erros repetitivos de copiar e colar
- ✓ Facilita a manutenção
- ✓ Deixa o foco na lógica de negócio

Boilerplate na prática

Abrir e fechar conexão JDBC

```
try {
    Connection c = DriverManager
        .getConnection(url, u, p);
    Statement st = c.createStatement();
    ResultSet rs = st.executeQuery(q);
    while (rs.next()) {
        System.out.print(rs.getInt(1));
    }
    rs.close();
    st.close();
    c.close();
}
catch (SQLException e) {
    e.printStackTrace();
}
```

Getters e setters de um POJO

```
public class Pessoa {
    private String nome;

    public String getNome() {
        return nome;
    }

    public void setNome(String n) {
        this.nome = n;
    }
}
```

- Lembre do record: ele elimina exatamente esse boilerplate.

Interfaces funcionais

Uma interface funcional tem exatamente um método abstrato. A lambda é a implementação desse método. O pacote `java.util.function` já traz as mais usadas.

Interface	Método	Recebe → devolve	Exemplo de lambda
<code>Predicate<T></code>	<code>test</code>	$T \rightarrow \text{boolean}$	<code>v -> v > 4</code>
<code>Function<T,R></code>	<code>apply</code>	$T \rightarrow R$	<code>e -> e.getNome()</code>
<code>Consumer<T></code>	<code>accept</code>	$T \rightarrow \text{nada}$	<code>v -> print(v)</code>
<code>Supplier<T></code>	<code>get</code>	$\text{nada} \rightarrow T$	<code>() -> new Aluno()</code>
<code>BinaryOperator<T></code>	<code>apply</code>	$T, T \rightarrow T$	<code>(a, b) -> a + b</code>

- É por isso que `filter()` aceita um `Predicate` e `map()` aceita uma `Function`: cada operação do `Stream` espera uma dessas interfaces.

A lambda é o corpo do método

Escrever a lambda equivale a implementar a interface — só que sem a cerimônia da classe anônima.

Classe anônima

```
Predicate<Integer> par =  
    new Predicate<Integer>() {  
        @Override  
        public boolean test(Integer v) {  
            return v % 2 == 0;  
        }  
    };
```

Lambda

```
Predicate<Integer> par =  
    v -> v % 2 == 0;
```

```
System.out.println(par.test(4));
```

● Run — saída no console
true

- As duas versões geram o mesmo comportamento. A lambda descarta 6 linhas de cerimônia e deixa apenas a regra: `v % 2 == 0`.

Method reference: o atalho ::

Quando a lambda só repassa o argumento para um método existente, o operador `::` substitui a expressão inteira.

Forma	Method reference	Lambda equivalente
Método estático	<code>Integer::parseInt</code>	<code>s -> Integer.parseInt(s)</code>
Método de um objeto	<code>System.out::println</code>	<code>v -> System.out.println(v)</code>
Método da instância	<code>Empregado::getNome</code>	<code>e -> e.getNome()</code>
Construtor	<code>Aluno::new</code>	<code>() -> new Aluno()</code>

```
lista.forEach(System.out::println);    // em vez de v -> System.out.println(v)
```

- `A::B` significa o método `B` da classe `A`. Use apenas quando a lambda não faz nada além de chamar esse método.

O que é um Stream?

Stream é uma sequência de elementos que pode ser processada em pipeline por meio de operações funcionais — como peças passando por uma esteira de produção.

Não é estrutura de dados

É uma visão sobre os dados, não um novo container como List ou Set.

É preguiçoso (lazy)

Só executa quando aparece uma operação terminal no fim do pipeline.

Pode ser paralelo

`parallelStream()` distribui o processamento entre vários núcleos.

- O Stream não altera a coleção de origem. Cada operação devolve um novo Stream — a lista original continua intacta.

O pipeline tem três partes

1

Fonte de dados

List, Set, array, Stream.of(...)

2

Operações intermediárias

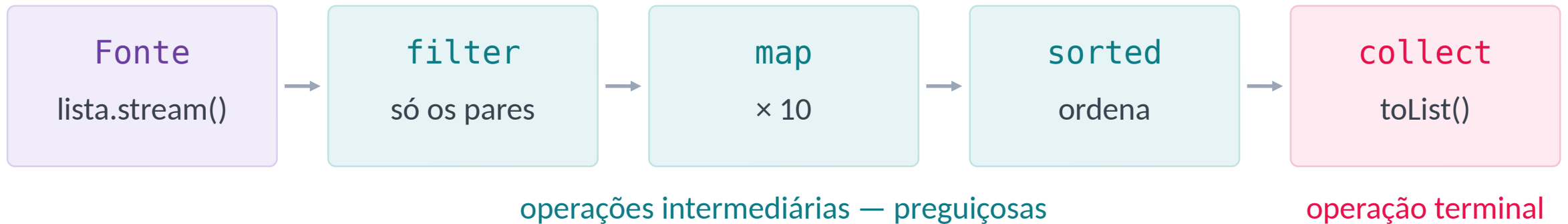
filter, map, sorted, distinct, limit — devolvem outro Stream e não executam de imediato

3

Operação terminal

collect, forEach, reduce, count, sum — dispara a execução e produz o resultado final

O pipeline em funcionamento



```
List<Integer> r = lista.stream()
    .filter(v -> v % 2 == 0)
    .map(v -> v * 10)
    .sorted()
    .collect(Collectors.toList());
```

STREAMS

Avaliação preguiçosa (lazy)

As operações intermediárias montam o pipeline, mas não processam nada. Sem operação terminal, nenhum elemento é percorrido.

Sem terminal: nada acontece

```
lista.stream()
    .filter(v -> {
        System.out.print(v);
        return v > 2;
    });
```

● Saída
(nada é impresso)

Com terminal: executa

```
lista.stream()
    .filter(v -> {
        System.out.print(v);
        return v > 2;
    })
    .count();
```

● Saída
1 2 3

Operações intermediárias

Operação	O que faz
<code>filter</code>	Mantém apenas os elementos que atendem a uma condição.
<code>distinct</code>	Mantém apenas os elementos únicos, descartando repetições.
<code>limit</code>	Mantém o número indicado de elementos, a partir do início.
<code>skip</code>	Descarta os primeiros elementos e segue com o restante.
<code>map</code>	Transforma cada elemento em um novo valor, possivelmente de outro tipo. O fluxo resultante tem a mesma quantidade de elementos.
<code>sorted</code>	Devolve os elementos em ordem. A quantidade não muda.

- Toda operação intermediária devolve outro Stream — por isso elas podem ser encadeadas em sequência.

Operações terminais

Operação	O que faz
<code>forEach</code>	Processa cada elemento do fluxo, por exemplo exibindo-o.
<code>count</code>	Devolve a quantidade de elementos do fluxo.
<code>average</code>	Calcula a média dos elementos de um fluxo numérico.
<code>max / min</code>	Localiza o maior e o menor valor do fluxo.
<code>reduce</code>	Reduz os elementos a um único valor usando uma função.
<code>collect</code>	Cria uma nova coleção com o resultado das operações anteriores.
<code>toArray</code>	Cria um array com o resultado das operações anteriores.

- Depois da operação terminal o Stream está consumido. Para processar de novo é preciso criar outro a partir da fonte.

IntStream

IntStream (pacote `java.util.stream`) é um Stream especializado em valores primitivos `int`. Existe para evitar o autoboxing/unboxing entre `int` e `Integer` — mais desempenho e menos consumo de memória.

IntStream

para o tipo `int`

sum, average, max, min, range

LongStream

para o tipo `long`

mesmas operações para
inteiros longos

DoubleStream

para o tipo `double`

mesmas operações para reais

- Não existe `FloatStream`: converta para `double` ou use `Stream<Float>`.

- A interface `Stream<T>` é usada para gerar fluxos de objetos.

Um problema para resolver

- Dado um conjunto de números inteiros, separe todos os números pares em uma lista. Em seguida, substitua cada número par pelo seu valor multiplicado por 10. Por fim, ordene a lista resultante em ordem crescente e exiba os valores finais.

■ dados de entrada

```
int[] x = {3, 10, 6, 1, 4, 8, 2, 5};
```

● Resultado esperado

20 40 60 80 100

Vamos resolver de duas formas:

1

Com laços tradicionais

2

Com IntStream

Solução com laços — parte 1

Separar os pares e multiplicar cada um por 10.

■ separar os pares

```
List<Integer> lista = new ArrayList<>();

for (int i : x) {
    if (i % 2 == 0) {
        lista.add(i);
    }
}
```

■ multiplicar por 10

```
for (int i = 0;
     i < lista.size(); i++) {
    lista.set(i,
              lista.get(i) * 10);
}
```

- Repare: dois laços, uma lista auxiliar, um índice e a lista original sendo alterada em cada passo.

Solução com laços — parte 2

Ordenar em ordem crescente e imprimir.

■ ordenar

```
for (int j = 0; j < lista.size(); j++) {  
    for (int i = 0; i < lista.size()-1; i++) {  
        if (lista.get(i) > lista.get(i+1)) {  
            int aux = lista.get(i);  
            lista.set(i, lista.get(i+1));  
            lista.set(i+1, aux);  
        }  
    }  
}
```

■ imprimir

```
for (int i : lista) {  
    System.out.print(  
        i + " ";  
    }  
}
```

- Ao todo: 4 laços aninhados, uma variável auxiliar e nenhuma linha que diga o que o programa quer fazer.

A mesma solução com IntStream

■ Funcional.java

```
1 int[] x = {3, 10, 6, 1, 4, 8, 2, 5};
2
3 IntStream.of(x)
4     .filter(valor -> valor % 2 == 0)    // separa os pares
5     .map(valor -> valor * 10)           // multiplica por 10
6     .sorted()                           // ordena
7     .forEach(valor -> System.out.print(valor + " "));
```

● Run — saída no console

20 40 60 80 100

- Quatro laços viraram quatro verbos: filtrar, transformar, ordenar e exibir. E a lista original nunca é alterada.

IntStream.of() e estatísticas

O método estático of() gera um fluxo de inteiros a partir de um array.

■ Estatisticas.java

```
1 int[] x = {1, 3, 5, 2, 0, 7};
2 IntStream s = IntStream.of(x);
3
4 // quantidade, máximo, mínimo e média
5 long quantidade = s.count();
6 int max = s.max().getAsInt();
7 int min = s.min().getAsInt();
8 double media = s.average().getAsDouble();
```



max(), min() e average() devolvem um Optional — daí o getAsInt() e o getAsDouble().



Atenção: esse código lança IllegalStateException — cada terminal consome o stream.

CUIDADO

Um Stream só pode ser usado uma vez

Depois da operação terminal o fluxo está fechado. Reutilizar a mesma variável provoca erro em tempo de execução.

Errado

```
IntStream s = IntStream.of(x);  
long q = s.count();  
int m = s.max().getAsInt();
```

- Erro em execução

IllegalStateException:
stream has already been
operated upon or closed

Certo

```
long q = IntStream.of(x).count();  
int m = IntStream.of(x)  
                .max()  
                .getAsInt();
```

- Alternativa: use `summaryStatistics()`, que devolve contagem, soma, mínimo, máximo e média em uma única passada.

STREAMS NUMÉRICOS

reduce(): somando os elementos

reduce() acumula os elementos dois a dois até sobrar um único valor.

Com laço

```
int a = 0;
for (int b : x) {
    a = somar(a, b);
}

public static int somar(int a,
                        int b) {
    return a + b;
}
```

Com reduce

```
int total = IntStream.of(x)
    .reduce(0, (a, b) -> a + b);

System.out.println(
    "Soma = " + total);
```

- O primeiro argumento é o valor inicial. A lambda recebe o acumulado e o próximo elemento. Para IntStream existe também o atalho sum().

STREAMS NUMÉRICOS

filter, sorted e forEach

Exibir os valores pares em ordem crescente.

Com laço

```
List<Integer> pares =  
    new ArrayList<>();  
for (int valor : x) {  
    if (verificarPar(valor)) {  
        pares.add(valor);  
    }  
}  
  
public static boolean  
    verificarPar(int valor) {  
    return valor % 2 == 0;  
}
```

Com Stream

```
IntStream.of(x)  
    .filter(valor -> valor % 2 == 0)  
    .sorted()  
    .forEach(v ->  
        System.out.print(v + " "));
```



A condição do if virou o argumento de filter(). O método auxiliar deixou de ser necessário.

Encadeando filter, map e sorted

Exibir os valores ímpares multiplicados por 10, em ordem crescente.

Com laço

```
for (int i = 0; i < x.length; i++) {  
    x[i] = multiplicar(x[i]);  
}  
  
public static int  
    multiplicar(int valor) {  
    return valor * 10;  
}
```

Com Stream

```
IntStream.of(x)  
    .filter(valor -> valor % 2 != 0)  
    .map(valor -> valor * 10)  
    .sorted()  
    .forEach(v ->  
        System.out.print(v + " "));
```

- A versão com laço altera o array original. A versão com Stream não toca em x — ela produz um novo fluxo de valores.

Stream de objetos: ordenar e coletar

■ Ordenar.java

```
1 Integer[] x = {2, 9, 5, 0, 3, 7, 1, 4, 8, 6};  
2 List<Integer> lista = Arrays.asList(x);  
3  
4 List<Integer> y = Arrays.stream(x)  
5     .sorted()  
6     .collect(Collectors.toList());
```

Arrays.asList(x)

método estático da classe
Arrays (java.util) que converte
um array em lista

Arrays.stream(x)

converte o array em um
Stream<Integer>

collect(...)

reduz o fluxo a uma estrutura
de resultado: List, Set, Map ou
String

Filtrar e coletar em uma nova lista

Filtrar os valores maiores que 4 e classificar em ordem crescente.

■ Filtrar.java

```
1 List<Integer> y = Arrays.stream(x)
2     .filter(valor -> valor > 4)
3     .sorted()
4     .collect(Collectors.toList());
5
6 System.out.println(y);
```

```
List<Integer> y = Arrays.stream(x)
    .sorted().toList();
```

● Run — saída no console
[5, 6, 7, 8, 9]

● A partir do Java 16 existe o atalho `.toList()`, que dispensa o `Collectors`. A lista devolvida, porém, é imutável.

A classe Empregado

Principal.java

```
1 Empregado[] empregado = {  
2     new Empregado("Antonio", "RH", 10000.00),  
3     new Empregado("Maria", "TI", 25000.00),  
4     new Empregado("Daniela", "Comunicacao", 35000.00),  
5     new Empregado("Pedro", "Financeiro", 15000.00)  
6 };
```

```
// gera uma lista a partir do vetor  
List<Empregado> lista = Arrays.asList(empregado);  
  
// impressão dos dados  
lista.stream().forEach(System.out::println);
```

- `stream()` converte a lista em um fluxo. `System.out::println` é um method reference: equivale a `v -> System.out.println(v)`.

Predicate: um filtro reutilizável

■ Filtro.java

```
1 // true para salários entre 5000 e 15000
2 Predicate<Empregado> filtroSalario =
3     e -> e.getSalario() >= 5000
4     && e.getSalario() <= 15000;
5
6 lista.stream()
7     .filter(filtroSalario)
8     .sorted(Comparator.comparing(
9         Empregado::getSalario))
10    .forEach(System.out::println);
```

- Predicate<T> é uma interface de java.util.function. Cria um filtro lógico que responde sim ou não.

- Guardar o Predicate em uma variável dá nome à regra de negócio e permite reaproveitá-la em outros pontos do programa.

Function e critérios de ordenação

■ Ordenacao.java

```
1 // functions para obter nome e departamento
2 Function<Empregado, String> nome =
3     Empregado::getNome;
4 Function<Empregado, String> depto =
5     Empregado::getDepartamento;
6
7 // critério de comparação
8 Comparator<Empregado> comparator =
9     Comparator.comparing(nome)
10     .thenComparing(depto);
11
12 lista.stream().sorted(comparator)
13     .forEach(System.out::println);
```

- `Function<T, R>` recebe um objeto do tipo T e devolve um valor do tipo R.

Para inverter a ordem:

```
.sorted(  
    comparator  
    .reversed())
```

Agrupando com groupingBy

■ Agrupar.java

```
1 Map<String, List<Empregado>> grupos = lista.stream()
2     .collect(Collectors.groupingBy(Empregado::getDepartamento));
3
4 grupos.forEach((dep, emps) -> {
5     System.out.println(dep);
6     emps.forEach(e -> System.out.println(e));
7 });
```

● Run — saída no console

TI

Nome = Maria | Salário = 25000,00

RH

Nome = Antonio | Salário = 10000,00

- groupingBy transforma o fluxo em um Map: a chave é o resultado da Function, o valor é a lista dos elementos daquele grupo.

Contando dentro de cada grupo

■ Contar.java

```
1 Map<String, Long> porDepartamento = lista.stream()  
2     .collect(Collectors.groupingBy(  
3         Empregado::getDepartamento,  
4         Collectors.counting()));  
5  
6 porDepartamento.forEach(  
7     (dep, qtd) -> System.out.println(dep + " tem " + qtd));
```

● Run — saída no console

```
TI tem 1  
RH tem 1  
Financeiro tem 1
```

● O segundo argumento de `groupingBy` é um downstream collector: em vez da lista, ele decide o que guardar em cada grupo.

Somando os salários

Com sum()

```
double total = lista.stream()
    .mapToDouble(
        Empregado::getSalario)
    .sum();
```

```
System.out.println(total);
```

- Run — saída no console
85000.0

Com reduce()

```
double total = lista.stream()
    .mapToDouble(
        Empregado::getSalario)
    .reduce(0, (a, b) -> a + b);
```

```
System.out.println(total);
```

- mapToDouble transforma cada elemento do fluxo em um double primitivo, liberando as operações numéricas.

- sum() é apenas um reduce() pronto — as duas versões produzem exatamente o mesmo resultado.

Calculando a média dos salários

Media.java

```
1 double media = lista.stream()
2     .mapToDouble(Empregado::getSalario)
3     .average()
4     .getAsDouble();
5
6 System.out.println(media);
```

Versão segura para fluxo vazio:

```
double media = lista.stream()
    .mapToDouble(Empregado::getSalario)
    .average()
    .orElse(0.0);
```

● Run — saída no console
21250.0

● average() devolve um OptionalDouble: se o fluxo estiver vazio não existe média. Por isso o getAsDouble().

Erros comuns e boas práticas



Esquecer a operação terminal

Sem collect, forEach ou reduce o pipeline nunca executa.



Reutilizar o mesmo Stream

Cada fluxo serve para uma única operação terminal.



Alterar a lista dentro do forEach

Isso reintroduz efeito colateral: prefira map e collect.



Usar Stream para tudo

Para um laço simples sobre poucos itens, o for continua mais claro.



Ignorar o Optional

max, min e average podem não ter resposta: use orElse.



Abusar do parallelStream()

Só compensa em volumes grandes e operações independentes.

Resumo: qual operação usar?

Preciso de...	Operação	Devolve
Selecionar alguns elementos	<code>filter</code>	Stream
Transformar cada elemento	<code>map</code> / <code>mapToInt</code>	Stream
Ordenar os elementos	<code>sorted</code>	Stream
Exibir cada elemento	<code>forEach</code>	nada
Montar uma nova coleção	<code>collect</code> / <code>toList</code>	List, Set, Map
Somar, contar ou tirar média	<code>sum</code> , <code>count</code> , <code>average</code>	número
Agrupar por um atributo	<code>Collectors.groupingBy</code>	Map

- Regra prática: intermediárias devolvem Stream e podem ser encadeadas; terminais encerram o pipeline e devolvem o resultado.

EXERCÍCIO 01

Nomes que começam com A

Dada uma lista com os nomes dos funcionários de uma empresa, escreva um programa em Java que execute as seguintes funções.

filter

1

Selecionar apenas os nomes que começam com a letra A

map

2

Transformar esses nomes em letras maiúsculas

collect

3

Exibir o resultado em uma nova lista

```
// dica: .filter(n -> n.startsWith("A")).map(String::toUpperCase)
```

EXERCÍCIO 02

Reajuste salarial

Você recebeu uma lista de objetos `Empregado` com nome, salário atual e anos de experiência. Aplique os reajustes conforme as regras abaixo.

5 anos ou mais de experiência

aumento de 20%

menos de 5 anos de experiência

aumento de 10%

- O cálculo dos novos salários deve usar a operação `map()` da API de Streams, gerando uma nova lista de empregados com os salários atualizados.
- Ao final, exiba os empregados com nome, novo salário e anos de experiência.

EXERCÍCIO 03

Produtos com desconto

Você recebeu uma lista de objetos Produto com os atributos nome (String), preco (double) e categoria (String). Implemente as funcionalidades a seguir.

filter

1

Selecionar apenas os produtos da categoria "Eletrônicos" com preço maior que 1000.0

map

2

Transformar em Strings no formato "NOME - Preço com desconto: X", aplicando 10% de desconto

collect

3

Exibir a nova lista resultante

```
// dica: String.format("%s - Preço com desconto: %.2f", nome, preco * 0.9)
```

EXERCÍCIO 04

Relatório de produtos (desafio)

Lista de Produto com nome, preco, categoria, avaliacao (0–5) e emEstoque (boolean).

1 Filtrar

Apenas produtos em estoque e com avaliação ≥ 4.0 .

2 Mapear

Nome em MAIÚSCULO e precoFinal: 15% de desconto em Eletrônicos, 10% nas demais categorias.

3 Agrupar

Por categoria, exibindo quantidade, soma, média, mínimo e máximo de precoFinal.

4 Ordenar

Em cada categoria, os nomes ordenados por precoFinal decrescente.

5 Ranquear

Top 3 produtos entre todas as categorias, por precoFinal decrescente.

- Dicas: summarizingDouble, groupingBy com downstream collector, sorted().limit(3).

Referências

1

DEITEL, H. M.; DEITEL, P. J. Java: como programar.
10ª edição. São Paulo: Prentice-Hall, 2010.

2

SCHILDT, H. Java para iniciantes.
6ª edição. Porto Alegre: Bookman, 2015.

3

KNUDSEN, J.; NIEMEYER, P. Aprendendo Java.
Rio de Janeiro: Elsevier Campus, 2000.

Referências

1

FLANAGAN, D. Java: o guia essencial.

Porto Alegre: Bookman, 2006.

2

ARNOLD, K.; GOSLING, J.; HOLMES, D. The Java programming language.

4th edition. Addison-Wesley, 2005.

3

JANDL JUNIOR, P. Introdução ao Java.

São Paulo: Berkeley, 2002.